Technical Guide: Building a Serverless RAG System on AWS with FAISS and Bedrock

joepareti54@gmail.com 4. Okt. 2026 17. Okt. 2026

Introduction and Purpose **3**
Update, October 2026 3
Background: the local Llama 2 prototype 3
Architecture Overview **4**
Part I: All Required Files **4**
Project Structure 5
Root Files 5
app.py 5
cdk.json 6
requirements.txt 6
requirements-dev.txt 7
.env.example 7
Config Directory 7
config/config.py 7
Stacks Directory 9
stacks/__init__.py 9
stacks/rag stack.py - modified in October 2026 9
Lambda Functions 12
lambdas/document_processor/document_processor.py 12
lambdas/document_processor/requirements.txt 15
lambdas/index_manager/index_manager.py 15
lambdas/index_manager/requirements.txt 17
lambdas/query_handler/query_handler.py 17
Query handler response 24
lambdas/query handler/requirements.txt 25
Layers Directory 25
layers/dependencies/requirements.txt 25
Scripts Directory 25
scripts/upload_documents_fixed.py 25
Test the script 33
Part II: Deployment Guide **33**
Prerequisites on Ubuntu PC 34
Step 1: Install Required Software 34
Step 2: Install AWS CDK 34

Step 3: Configure AWS Credentials 34
Prerequisites in AWS Console 35
Step 4: Enable Bedrock Model Access 35
Create the Project 35
Step 1: Create Directory Structure 35
Step 2: Create All Files 35
Step 3: Create requirements.txt Files 36
Step 4: Install Lambda Layer Dependencies 36
Step 5: Set Up Python Virtual Environment 37
Step 6: Set Environment Variables 37
Step 7: Configure the LLM Model ID 37
Deploy the Solution 38
Step 1: Bootstrap CDK (first time only) 38
Step 2: Synthesize the CloudFormation Template 38
Step 3: Deploy the Stack 38
Step 4: Verify 38
Upload Documents and Build the Index 39
Step 1: Get Bucket Name 39
Step 2: Upload Test Document 39
Step 3: Process the Document 39
Step 4: Build the FAISS Index 40
Step 4.5: Retrieve the API Key 40
Step 5: Test Query 41
Upload PDF Documents 41
Step 1: Install Script Dependencies 41
Step 2: Upload Script 41
Step 3: Upload Documents 42
Step 4: Rebuild the Index 42
Step 5: Verify the Index 43
Updating dependencies and re-ingesting the corpus. 43
Commands and scripts 43
Troubleshooting 44
Lambda Layer 44
PDF Parsing 45
When uploading files to S3, metadata must be ASCII 45
S3 event notifications are not retroactive 45
Concurrency throttling under bulk ingest 45
403 Forbidden on /query 46
Indexing and S3 List Pagination 46
CLI Flag 47
Running Test Queries **47**
Archegos and Credit Suisse 47
Private Credit - Irrational Exuberance 49
Cost **51**

Results and Discussion **52**
Github 52
Retrieval Quality and Groundedness 52
Comparison with the Local Llama 2 Prototype 52
Features Proposed but not implemented 53
Toward Agentic AI in AWS 53
Limitations 53

Introduction and Purpose

This guide consolidates the design, implementation, debugging, and operational lessons learned while building a production-style Retrieval-Augmented Generation (RAG) system on AWS. The system ingests PDF documents (in this case, a corpus of approximately 3,300 financial press articles spanning 2020–2026), processes them into embeddings using Amazon Bedrock's Titan model, stores a FAISS vector index in Amazon S3, and answers natural-language questions through an API Gateway endpoint backed by Anthropic Claude on Bedrock.

This revision incorporates the pypdf upgrade following observed PDF text-extraction corruption, the platform-compatible Lambda layer build procedure, corpus reprocessing and FAISS index recovery instructions, and enhancements to query responses for inspecting retrieved chunks, extracted facts, and information gaps.

Update, October 2026

The deployed model has been changed from Claude Sonnet 4.5 to Claude Haiku 4.5 due to end of support situation by AWS , and following validation runs that showed equivalent output quality at lower cost and latency. References to Sonnet 4.5 in the body text below reflect the original deployment; the current model ID is in `stacks/rag_stack.py`.

Background: the local Llama 2 prototype

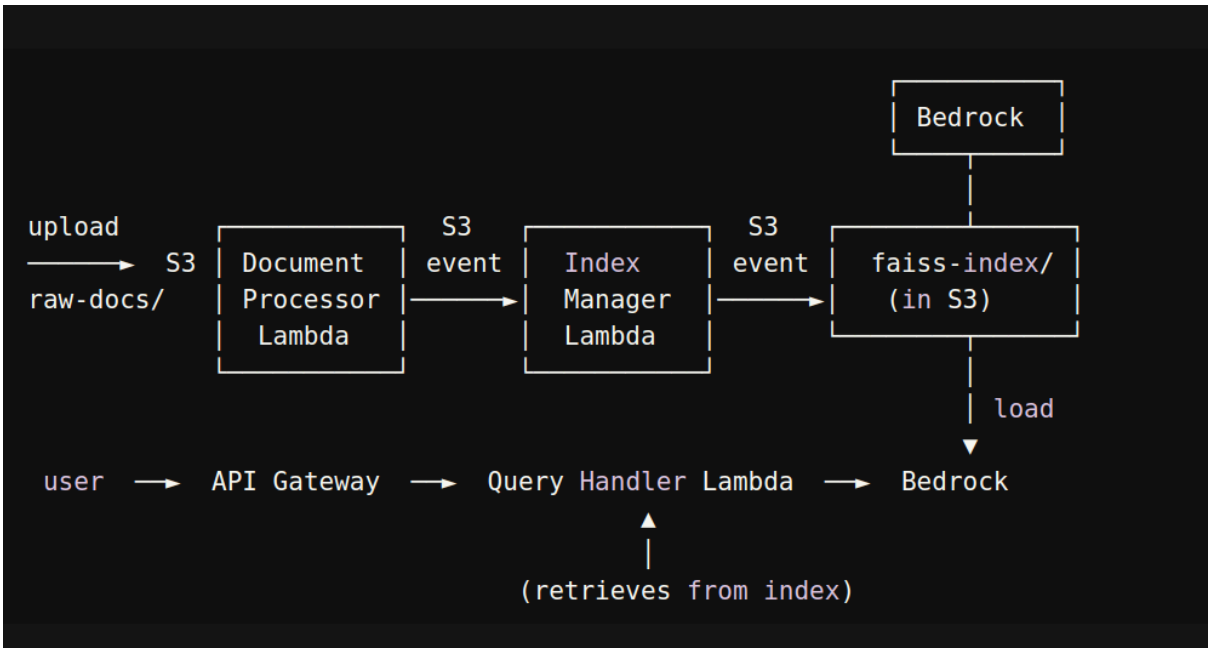
This AWS system is the cloud-native successor to an earlier local prototype (Pareti, March 2025) that implemented the same RAG pattern entirely on a Ubuntu workstation with an Nvidia GPU and 6 GB of VRAM. The prototype used PDF text extraction, embedding into a FAISS index, top-k semantic retrieval, and a two-step prompt strategy that first summarises the retrieved chunks and then generates a final answer combining document context with the model's general knowledge — but it did so under hardware constraints that conditioned

subsequent decisions. The **Memory limit** was critical and conditioned design choices, even running Llama 2 7B in 4-bit quantization:

- a low dimension sentence transformer for embeddings;
- only the first 1000 characters of each document were indexed;
- lower output quality, inherent to a relatively small LLM compared to state-of-the-art models.

Further details are available in the original prototype document.

Architecture Overview



Part I: All Required Files

Project Structure

```
rag-solution/
├── app.py
├── cdk.json
├── requirements.txt
├── config/
│   └── config.py
├── stacks/
│   ├── __init__.py
│   └── rag_stack.py
├── scripts/
│   ├── upload_documents_fixed.py
│   └── test_query.py
├── lambdas/
│   ├── document_processor/
│   │   └── document_processor.py
│   ├── index_manager/
│   │   └── index_manager.py
│   └── query_handler/
│       └── query_handler.py
└── layers/
    └── dependencies/
        ├── requirements.txt
        └── python/
            ├── faiss/
            ├── numpy/
            └── pypdf/
```

Root Files

app.py

```
#!/usr/bin/env python3
import os
import aws_cdk as cdk
from stacks.rag_stack import RAGStack

app = cdk.App()
```

```
RAGStack(
    app,
    "RAGStack",
    env=cdk.Environment(
        account=os.getenv("CDK_DEFAULT_ACCOUNT"),
        region=os.getenv("CDK_DEFAULT_REGION", "us-east-1")
    ),
    description="RAG Solution with FAISS vector store and Bedrock"
)

app.synth()
```

cdk.json

```
{
  "app": "python3 app.py",
  "watch": {
    "include": ["**"],
    "exclude": [
      "README.md",
      "cdk*.json",
      "requirements*.txt",
      "**/__pycache__",
      "**/.pytest_cache",
      "**/*.pyc",
      ".git",
      ".venv",
      "datasets"
    ]
  },
  "context": {
    "@aws-cdk/aws-lambda:recognizeLayerVersion": true,
    "@aws-cdk/core:stackRelativeExports": true,
    "@aws-cdk/aws-apigateway:usagePlanKeyOrderInsensitiveId": true,
    "@aws-cdk/core:enablePartitionLiterals": true,
    "@aws-cdk/aws-iam:minimizePolicies": true,
    "@aws-cdk/core:validateSnapshotRemovalPolicy": true
  }
}
```

requirements.txt

```
aws-cdk-lib==2.170.0
constructs>=10.0.0,<11.0.0
python-dotenv==1.0.0
boto3==1.34.0
```

requirements-dev.txt

```
-r requirements.txt
pytest==7.4.0
pytest-cov==4.1.0
moto==4.2.0
```

.env.example

```
CDK_DEFAULT_ACCOUNT=123456789012
CDK_DEFAULT_REGION=us-east-1
```

Config Directory

config/config.py

The system uses FAISS `IndexFlatIP` for exact inner-product search. Flat indexing is optimal for corpora under ~100k vectors, where exhaustive search is fast enough and recall is perfect. `IVFFlat` is noted as a future option for larger deployments.

```
from dataclasses import dataclass


@dataclass
class LambdaConfig:
    document_processor_memory: int = 1024
    document_processor_timeout: int = 300
```

```
    index_manager_memory: int = 3008  # FAISS needs more memory
    index_manager_timeout: int = 600
    query_handler_memory: int = 3008
    query_handler_timeout: int = 30


@dataclass
class BedrockConfig:
    embedding_model_id: str = "amazon.titan-embed-text-v2:0"
    llm_model_id: str = "anthropic.claude-3-sonnet-20240229-v1:0"
    embedding_dimension: int = 1024


@dataclass
class ChunkingConfig:
    chunk_size: int = 1000
    chunk_overlap: int = 200


@dataclass
class FAISSConfig:
    index_file: str = "faiss_index.bin"
    metadata_file: str = "metadata.json"
    index_type: str = "IVFFlat"  # Options: Flat, IVFFlat, HNSW
    nlist: int = 100  # Number of clusters for IVFFlat


@dataclass
class RAGConfig:
    lambda_config: LambdaConfig
    bedrock: BedrockConfig
    chunking: ChunkingConfig
    faiss: FAISSConfig

    @classmethod
    def default(cls) -> "RAGConfig":
        return cls(
            lambda_config=LambdaConfig(),
            bedrock=BedrockConfig(),
            chunking=ChunkingConfig(),
            faiss=FAISSConfig()
        )


def get_config() -> RAGConfig:
    return RAGConfig.default()
```

Stacks Directory

stacks/__init__.py

```
from .rag_stack import RAGStack

__all__ = ["RAGStack"]
```

stacks/rag stack.py - modified in October 2026

```
import aws_cdk.aws_s3_notifications as s3n
from aws_cdk import (
    Stack,
    Duration,
    RemovalPolicy,
    CfnOutput,
    aws_s3 as s3,
    aws_lambda as lambda_,
    aws_iam as iam,
    aws_apigateway as apigw,
)
from constructs import Construct

class RAGStack(Stack):
    def __init__(self, scope: Construct, construct_id: str, **kwargs) -> None:
        super().__init__(scope, construct_id, **kwargs)

        # S3 Bucket for documents and FAISS index
        self.bucket = s3.Bucket(
            self, "RAGBucket",
            removal_policy=RemovalPolicy.DESTROY,
            auto_delete_objects=True,
            versioned=True,
            encryption=s3.BucketEncryption.S3_MANAGED,
        )

        # Lambda Layer for shared dependencies
```

```
        dependencies_layer = lambda_.LayerVersion(
            self, "DependenciesLayer",
            code=lambda_.Code.from_asset("layers/dependencies"),
            compatible_runtimes=[lambda_.Runtime.PYTHON_3_9],
            description="FAISS, numpy, and other dependencies",
        )

        # Common environment variables
        common_env = {
            "BUCKET_NAME": self.bucket.bucket_name,
            "EMBEDDING_MODEL_ID": "amazon.titan-embed-text-v2:0",
#             "LLM_MODEL_ID": "us.anthropic.claude-3-5-haiku-20241022-v1:0",
#             "LLM_MODEL_ID": "us.anthropic.claude-sonnet-4-5-20250929-v1:0",
            "LLM_MODEL_ID": "us.anthropic.claude-haiku-4-5-20251001-v1:0",
            "INDEX_PREFIX": "faiss-index/",
        }

        # IAM Policy for Bedrock
        bedrock_policy = iam.PolicyStatement(
            actions=[
                "bedrock:InvokeModel",
                "bedrock:InvokeModelWithResponseStream",
            ],
            resources=["*"],
        )

        # Document Processor Lambda
        self.document_processor = lambda_.Function(
            self, "DocumentProcessor",
            runtime=lambda_.Runtime.PYTHON_3_9,
            handler="document_processor.handler",
            code=lambda_.Code.from_asset("lambdas/document_processor"),
            timeout=Duration.minutes(5),
            memory_size=1024,
            environment=common_env,
            layers=[dependencies_layer],
        )
        self.document_processor.add_to_role_policy(bedrock_policy)
        self.bucket.grant_read_write(self.document_processor)
        # Trigger Document Processor when files are uploaded
#
self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED,s3n.LambdaDestin
ation(self.document_processor),s3n.LambdaDestination(self.document_processor),s3.Noti
ficationKeyFilter(prefix="uploads/"))
        self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED,
s3n.LambdaDestination(self.document_processor),
```

```
s3.NotificationKeyFilter(prefix="uploads/"))

        # Index Manager Lambda
        self.index_manager = lambda_.Function(
            self, "IndexManager",
            runtime=lambda_.Runtime.PYTHON_3_9,
            handler="index_manager.handler",
            code=lambda_.Code.from_asset("lambdas/index_manager"),
            timeout=Duration.minutes(10),
            memory_size=3008,
            environment=common_env,
            layers=[dependencies_layer],
        )
        self.index_manager.add_to_role_policy(bedrock_policy)
        self.bucket.grant_read_write(self.index_manager)

        # Query Handler Lambda
        self.query_handler = lambda_.Function(
            self, "QueryHandler",
            runtime=lambda_.Runtime.PYTHON_3_9,
            handler="query_handler.handler",
            code=lambda_.Code.from_asset("lambdas/query_handler"),
            timeout=Duration.seconds(30),
            memory_size=3008,
            environment=common_env,
            layers=[dependencies_layer],
#           reserved_concurrent_executions=2,
        )
        self.query_handler.add_to_role_policy(bedrock_policy)
        self.bucket.grant_read(self.query_handler)

        # API Gateway
        api = apigw.RestApi(
            self, "RAGAPI",
            rest_api_name="RAG Service",
            description="RAG Query API",
            deploy_options=apigw.StageOptions(stage_name="prod"),
        )

        query_resource = api.root.add_resource("query")
        query_resource.add_method(
            "POST",
            apigw.LambdaIntegration(self.query_handler),
            api_key_required=True,
        )
        api_key = api.add_api_key("RAGApiKey", api_key_name="rag-default-key",
```

```
description="Default key for RAG API access",)
        usage_plan =
api.add_usage_plan("RAGUsagePlan",name="rag-default-plan",throttle=apigw.ThrottleSet
tings(rate_limit=2,burst_limit=5,),quota=apigw.QuotaSettings(limit=1000,period=apigw.Peri
od.DAY,))

        usage_plan.add_api_key(api_key)
        usage_plan.add_api_stage(stage=api.deployment_stage)
        # Outputs
        CfnOutput(self, "BucketName", value=self.bucket.bucket_name)
        CfnOutput(self, "APIEndpoint", value=api.url)
        CfnOutput(self, "DocumentProcessorArn",
value=self.document_processor.function_arn)
        CfnOutput(self, "IndexManagerArn", value=self.index_manager.function_arn)
        CfnOutput(self, "ApiKeyId", value=api_key.key_id, description="Use 'aws apigateway
get-api-key --api-key <id> --include-value' to retrieve",)
```

Lambda Functions

lambdas/document_processor/document_processor.py

```
import json
import os
import boto3
import re
import io
from urllib.parse import unquote_plus

s3 = boto3.client("s3")
bedrock = boto3.client("bedrock-runtime")

BUCKET_NAME = os.environ["BUCKET_NAME"]
EMBEDDING_MODEL_ID = os.environ.get("EMBEDDING_MODEL_ID",
"amazon.titan-embed-text-v2:0")
CHUNK_SIZE = 1000
CHUNK_OVERLAP = 200

def chunk_text(text, chunk_size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
    chunks = []
    start = 0
```

```
    text = re.sub(r'\s+', ' ', text).strip()

    while start < len(text):
        end = start + chunk_size
        if end < len(text):
            last_period = text.rfind('.', start, end)
            last_newline = text.rfind('\n', start, end)
            break_point = max(last_period, last_newline)
            if break_point > start:
                end = break_point + 1

        chunk = text[start:end].strip()
        if chunk:
            chunks.append(chunk)
        start = end - overlap

    return chunks

def get_embedding(text):
    response = bedrock.invoke_model(
        modelId=EMBEDDING_MODEL_ID,
        body=json.dumps({"inputText": text}),
        contentType="application/json",
    )
    result = json.loads(response["body"].read())
    return result["embedding"]

def handler(event, context):
    print(f"Received event: {json.dumps(event)}")

    try:
        # Handle S3 event notification format
        if "Records" in event:
            record = event["Records"][0]
            bucket = record["s3"]["bucket"]["name"]
            key = unquote_plus(record["s3"]["object"]["key"])
        else:
            # Direct invocation format
            bucket = event.get("bucket", BUCKET_NAME)
            key = event["key"]

        print(f"Processing file: s3://{bucket}/{key}")

        # Skip non-PDF files
        if not key.lower().endswith('.pdf'):
            print(f"Skipping non-PDF file: {key}")
```

```
        return {
            "statusCode": 200,
            "body": json.dumps({"message": "Skipped non-PDF file", "key": key}),
        }

    # Download PDF
    response = s3.get_object(Bucket=bucket, Key=key)
    content = response["Body"].read()
    print(f"Downloaded {len(content)} bytes")

    # Extract text from PDF using pypdf
    from pypdf import PdfReader
    pdf_reader = PdfReader(io.BytesIO(content))
    text = ""
    for page in pdf_reader.pages:
        page_text = page.extract_text()
        if page_text:
            text += page_text + "\n"
    print(f"Extracted {len(text)} characters from PDF")

    if not text.strip():
        print(f"No text extracted from PDF: {key}")
        return {
            "statusCode": 200,
            "body": json.dumps({"message": "No text extracted from PDF", "key": key}),
        }

    chunks = chunk_text(text)
    print(f"Created {len(chunks)} chunks")

    processed_chunks = []
    for i, chunk in enumerate(chunks):
        embedding = get_embedding(chunk)
        processed_chunks.append({
            "chunk_id": f"{key}_{i}",
            "text": chunk,
            "embedding": embedding,
            "source": key,
            "chunk_index": i,
        })

    output_key = f"processed/{key.replace('/', '_')}.json"
    s3.put_object(
        Bucket=bucket,
        Key=output_key,
        Body=json.dumps(processed_chunks),
```

```
            ContentType="application/json",
        )
        print(f"Saved processed chunks to {output_key}")

        return {
            "statusCode": 200,
            "body": json.dumps({
                "message": "Document processed successfully",
                "chunks_created": len(processed_chunks),
                "output_key": output_key,
            }),
        }
    except Exception as e:
        print(f"Error processing document: {str(e)}")
        import traceback
        traceback.print_exc()
        raise
```

lambdas/document_processor/requirements.txt

```
# Dependencies provided by layer
```

lambdas/index_manager/index_manager.py

```
import json
import os
import boto3
import numpy as np
import faiss
import tempfile

s3 = boto3.client("s3")

BUCKET_NAME = os.environ["BUCKET_NAME"]
INDEX_PREFIX = os.environ.get("INDEX_PREFIX", "faiss-index/")

def handler(event, context):
    try:
        action = event.get("action", "build")
```

```
        if action == "build":
            return build_index(event)
        elif action == "update":
            return update_index(event)
        else:
            return {"statusCode": 400, "body": json.dumps({"error": "Invalid action"})}
    except Exception as e:
        return {"statusCode": 500, "body": json.dumps({"error": str(e)})}

def build_index(event):
    processed_prefix = event.get("processed_prefix", "processed/")

    all_embeddings = []
    all_metadata = []

    # Paginate through all objects
    paginator = s3.get_paginator('list_objects_v2')
    pages = paginator.paginate(Bucket=BUCKET_NAME, Prefix=processed_prefix)

    for page in pages:
        for obj in page.get("Contents", []):
            key = obj["Key"]
            if key.endswith(".json"):
                data = s3.get_object(Bucket=BUCKET_NAME, Key=key)
                chunks = json.loads(data["Body"].read().decode("utf-8"))

                for chunk in chunks:
                    all_embeddings.append(chunk["embedding"])
                    all_metadata.append({
                        "chunk_id": chunk["chunk_id"],
                        "text": chunk["text"],
                        "source": chunk["source"],
                        "chunk_index": chunk["chunk_index"],
                    })

    if not all_embeddings:
        return {"statusCode": 400, "body": json.dumps({"error": "No embeddings found"})}

    embeddings_array = np.array(all_embeddings, dtype=np.float32)
    dimension = embeddings_array.shape[1]

    index = faiss.IndexFlatIP(dimension)
    faiss.normalize_L2(embeddings_array)
    index.add(embeddings_array)
```

```
    with tempfile.TemporaryDirectory() as tmpdir:
        index_path = os.path.join(tmpdir, "index.faiss")
        metadata_path = os.path.join(tmpdir, "metadata.json")

        faiss.write_index(index, index_path)
        with open(metadata_path, "w") as f:
            json.dump(all_metadata, f)

        s3.upload_file(index_path, BUCKET_NAME, f"{INDEX_PREFIX}index.faiss")
        s3.upload_file(metadata_path, BUCKET_NAME, f"{INDEX_PREFIX}metadata.json")

    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "Index built successfully",
            "total_vectors": len(all_embeddings),
            "dimension": dimension,
        }),
    }

def update_index(event):
    return build_index(event)
```

lambdas/index_manager/requirements.txt

```
# Dependencies provided by layer
```

lambdas/query_handler/query_handler.py

```
# new version August 2026 - with chunk-to-source mapping
# see  https://poe.com/chat/66tmdly17ajaz0hs7k

import json
import os
import boto3
import numpy as np
import faiss
import tempfile

s3 = boto3.client("s3")
```

```
bedrock = boto3.client("bedrock-runtime")

BUCKET_NAME = os.environ["BUCKET_NAME"]
INDEX_PREFIX = os.environ.get("INDEX_PREFIX", "faiss-index/")
EMBEDDING_MODEL_ID = os.environ.get("EMBEDDING_MODEL_ID",
"amazon.titan-embed-text-v2:0")
LLM_MODEL_ID = os.environ.get("LLM_MODEL_ID",
"anthropic.claude-3-sonnet-20240229-v1:0")

index_cache = {}


# ============================================================
# GROUNDED RAG SYSTEM PROMPT
# ============================================================
GROUNDED_RAG_SYSTEM_PROMPT = """You are a research assistant answering
questions using ONLY the retrieved document chunks provided below.

## GROUNDING RULES (CRITICAL)

1. **EXTRACTION BEFORE DISCLAIMER**: Before stating that information is "not
available" or "not in the context," you MUST:
   - Read each chunk COMPLETELY, not just headlines or first sentences
   - Look for the requested information expressed in DIFFERENT TERMINOLOGY
   - Check for relevant facts EMBEDDED in narrative paragraphs
   - Consider synonyms and related concepts (e.g., "margin" includes: collateral, haircut,
margin call, margin requirement, initial margin, variation margin)

2. **PARTIAL RELEVANCE RULE**: If chunks contain SOME relevant information but not
all:
   - Extract and present what IS available with specific facts and figures
   - State SPECIFICALLY what is missing (e.g., "The chunks provide the margin reduction
percentage but not the exact dates")
   - NEVER use blanket disclaimers like "no information available" when partial information
exists

3. **TERMINOLOGY FLEXIBILITY**: Query terms may not match document vocabulary
exactly:
   - "Margins" → margin requirements, collateral, margin calls, haircuts, collateral return
   - "Leverage" → exposure, gross notional, debt-to-equity, concentration limits
   - Look for the CONCEPT, not just the exact WORD

4. **QUANTITATIVE EXTRACTION**: When documents contain specific numbers,
percentages, or dollar amounts related to the query, you MUST extract and report them.

## OUTPUT FORMAT
```

```
You must return valid JSON with this exact structure:
{{
  "answer": "Your comprehensive answer here, with specific facts and figures from the chunks",
  "sources_used": [
    {{
      "chunk_id": "CHUNK_0",
      "facts_extracted": ["specific fact 1", "specific fact 2"]
    }}
  ],
  "retrieval_status": "sources_used",
  "information_gaps": ["specific item 1 not found in any chunk", "specific item 2 not found"]
}}

The retrieval_status field must be one of:
- "sources_used" — at least one chunk contributed to the answer
- "sources_retrieved_but_irrelevant" — chunks were retrieved but none relevant to query
- "no_sources_retrieved" — no chunks provided (system error)

## RETRIEVED CHUNKS

{chunks}

## USER QUERY

{query}

Return ONLY the JSON object, no other text."""


def get_embedding(text):
    response = bedrock.invoke_model(
        modelId=EMBEDDING_MODEL_ID,
        body=json.dumps({"inputText": text}),
        contentType="application/json",
    )
    result = json.loads(response["body"].read())
    return result["embedding"]


def load_index():
    if "index" in index_cache and "metadata" in index_cache:
        return index_cache["index"], index_cache["metadata"]

    with tempfile.TemporaryDirectory() as tmpdir:
```

```
        index_path = os.path.join(tmpdir, "index.faiss")
        metadata_path = os.path.join(tmpdir, "metadata.json")

        s3.download_file(BUCKET_NAME, f"{INDEX_PREFIX}index.faiss", index_path)
        s3.download_file(BUCKET_NAME, f"{INDEX_PREFIX}metadata.json",
metadata_path)

        index = faiss.read_index(index_path)
        with open(metadata_path, "r") as f:
            metadata = json.load(f)

    index_cache["index"] = index
    index_cache["metadata"] = metadata
    return index, metadata


def search(query, k=5):
    index, metadata = load_index()
    query_embedding = np.array([get_embedding(query)], dtype=np.float32)
    faiss.normalize_L2(query_embedding)

    distances, indices = index.search(query_embedding, k)

    results = []
    for i, idx in enumerate(indices[0]):
        if idx < len(metadata):
            result = metadata[idx].copy()
            result["score"] = float(distances[0][i])
            results.append(result)

    return results


def format_chunks_with_ids(chunks):
    """Format chunks with explicit IDs for attribution tracking."""
    formatted = []
    for i, chunk in enumerate(chunks):
        chunk_id = f"CHUNK_{i}"
        source_name = chunk.get('source', 'unknown')

        # Show FULL source path for better traceability
        formatted.append(f"""--- {chunk_id} ---
Source: {source_name}
Score: {chunk['score']:.3f}
Content:
{chunk['text']}
```

```
--- END {chunk_id} ---""")

    return "\n\n".join(formatted)


def generate_response(query, context_chunks):
    """Generate response with grounding enforcement and structured output."""

    formatted_chunks = format_chunks_with_ids(context_chunks)

    prompt = GROUNDED_RAG_SYSTEM_PROMPT.format(
        chunks=formatted_chunks,
        query=query
    )

    response = bedrock.invoke_model(
        modelId=LLM_MODEL_ID,
        body=json.dumps({
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": 4096,
            "messages": [{"role": "user", "content": prompt}],
        }),
        contentType="application/json",
    )

    result = json.loads(response["body"].read())
    response_text = result["content"][0]["text"]

    # Parse the structured response
    try:
        # Handle case where model wraps JSON in markdown code blocks
        parse_text = response_text
        if "```json" in parse_text:
            json_start = parse_text.find("```json") + 7
            json_end = parse_text.find("```", json_start)
            parse_text = parse_text[json_start:json_end].strip()
        elif "```" in parse_text:
            json_start = parse_text.find("```") + 3
            json_end = parse_text.find("```", json_start)
            parse_text = parse_text[json_start:json_end].strip()

        structured_response = json.loads(parse_text)

        # Validate required fields exist
        if "answer" not in structured_response:
            structured_response["answer"] = response_text
```

```
        if "retrieval_status" not in structured_response:
            structured_response["retrieval_status"] = "sources_used"
        if "sources_used" not in structured_response:
            structured_response["sources_used"] = []
        if "information_gaps" not in structured_response:
            structured_response["information_gaps"] = []

    except json.JSONDecodeError:
        # Fallback: treat entire response as answer
        structured_response = {
            "answer": response_text,
            "sources_used": [],
            "retrieval_status": "sources_used",
            "information_gaps": [],
            "_parse_error": "Response was not valid JSON, using raw text"
        }

    return structured_response


def handler(event, context):
    try:
        body = json.loads(event.get("body", "{}"))
        query = body.get("query")
        k = body.get("k", 5)

        if not query:
            return {
                "statusCode": 400,
                "body": json.dumps({"error": "Query is required"}),
            }

        search_results = search(query, k)

        if not search_results:
            return {
                "statusCode": 200,
                "headers": {"Content-Type": "application/json"},
                "body": json.dumps({
                    "query": query,
                    "answer": "No relevant documents were retrieved for this query.",
                    "sources": [],
                    "retrieval_status": "no_sources_retrieved",
                    "sources_used": [],
                    "information_gaps": [query]
                }),
```

```
            }

        structured_response = generate_response(query, search_results)

        # Build a mapping from CHUNK_X to source filename
        chunk_id_to_source = {f"CHUNK_{i}": r["source"] for i, r in
enumerate(search_results)}

        # Enrich sources_used with actual filenames
        enriched_sources_used = []
        for src in structured_response.get("sources_used", []):
            chunk_id = src.get("chunk_id", "")
            enriched_sources_used.append({
                "chunk_id": chunk_id,
                "source_file": chunk_id_to_source.get(chunk_id, "unknown"),
                "facts_extracted": src.get("facts_extracted", [])
            })

        response_body = {
            "query": query,
            "answer": structured_response.get("answer", ""),
            "sources": [
                {
                    "index": i,
                    "chunk_id": f"CHUNK_{i}",
                    "source": r["source"],
                    "score": r["score"]
#                     "text": r["text"]
                }
                for i, r in enumerate(search_results)
            ],
            "retrieval_status": structured_response.get("retrieval_status", "sources_used"),
            "sources_used": enriched_sources_used,
            "information_gaps": structured_response.get("information_gaps", []),
        }

        if "_parse_error" in structured_response:
            response_body["_parse_error"] = structured_response["_parse_error"]

        return {
            "statusCode": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps(response_body),
        }

    except Exception as e:
```

```
    import traceback
    return {
        "statusCode": 500,
        "body": json.dumps({
            "error": str(e),
            "trace": traceback.format_exc()
        }),
    }
```

Query handler response

the query handler returns a single JSON object with the following top-level fields. All fields are always present; `sources`, `sources_used`, and `information_gaps` may be empty arrays.

Field	Type	Always present	Meaning
`query`	string	yes	The original query string as received by the handler. Echoed unchanged.
`answer`	string	yes	Natural-language answer produced from the grounded prompt. May be a refusal if `retrieval_status` is not `sources_used`.
`sources`	array of objects	yes (possibly empty)	Every chunk returned by the FAISS search, in retrieval order. See **Source entry** below.
`retrieval_status`	enum string	yes	One of `sources_used`, `sources_retrieved_but_irrelevant`, `no_sources_retrieved`. See **Retrieval status** below.
`sources_used`	array of objects	yes (possibly empty)	Subset of `sources` the model actually drew facts from, with the extracted facts. See **Used-source entry** below.
`information_gaps`	array of strings	yes (possibly empty)	Parts of the query the model could not answer from the retrieved context.

{
 "query": "...",
 "answer": "...",
 "sources": [
 {"index": 0, "chunk_id": "CHUNK_0", "source": "...", "score": 0.58, "text": "..."}
 ],
 "retrieval_status": "sources_used" | "sources_retrieved_but_irrelevant" |
"no_sources_retrieved",

```
"sources_used": [
{"chunk_id": "CHUNK_0", "source_file": "...", "facts_extracted": ["fact 1", "fact 2"]}
],
"information_gaps": ["item not found in any chunk", ...]
}
```

lambdas/query handler/requirements.txt

```
# Dependencies provided by layer
```

Layers Directory

layers/dependencies/requirements.txt

```
faiss-cpu==1.7.4
numpy==1.26.4
pypdf>=5.0.0
```

```
layers/dependencies/
├── requirements.txt
└── python/
    ├── faiss/          # Vector similarity search
    ├── numpy/          # Numerical operations
    └── pypdf/          # PDF text extraction
```

Scripts Directory

scripts/upload_documents_fixed.py

```
#!/usr/bin/env python3
"""
Script to bulk upload PDF documents to the RAG system.
Uploads to a FLAT S3 structure (all files in uploads/ folder).

Usage:
```

```
    python upload_documents.py /path/to/datasets
    python upload_documents.py /path/to/datasets --dry-run
    python upload_documents.py /path/to/datasets --year 2024
    python upload_documents.py /path/to/datasets --from-year 2023 --to-year 2025
"""

import os
import sys
import boto3
import argparse
from pathlib import Path
from collections import defaultdict
import time
import uuid


VALID_YEARS = [str(year) for year in range(2020, 2027)]

MONTH_NAMES = {
    'january': '01', 'jan': '01', '01': '01',
    'february': '02', 'feb': '02', '02': '02',
    'march': '03', 'mar': '03', '03': '03',
    'april': '04', 'apr': '04', '04': '04',
    'may': '05', '05': '05',
    'june': '06', 'jun': '06', '06': '06',
    'july': '07', 'jul': '07', '07': '07',
    'august': '08', 'aug': '08', '08': '08',
    'september': '09', 'sep': '09', 'sept': '09', '09': '09',
    'october': '10', 'oct': '10', '10': '10',
    'november': '11', 'nov': '11', '11': '11',
    'december': '12', 'dec': '12', '12': '12',
}

MONTH_DISPLAY = {
    '01': 'january', '02': 'february', '03': 'march', '04': 'april',
    '05': 'may', '06': 'june', '07': 'july', '08': 'august',
    '09': 'september', '10': 'october', '11': 'november', '12': 'december',
}


def sanitize_metadata(text):
    """Convert text to ASCII-safe string for S3 metadata."""
    return text.encode('ascii', 'replace').decode('ascii')


def get_bucket_name():
```

```
    """Get the documents bucket name from CloudFormation outputs or environment."""
    # Check environment variable first
    bucket = os.environ.get('DOCUMENTS_BUCKET')
    if bucket:
        return bucket

    cf_client = boto3.client("cloudformation")
    try:
        response = cf_client.describe_stacks(StackName="RAGStack")
        outputs = response["Stacks"][0]["Outputs"]
        for output in outputs:
            if output["OutputKey"] == "BucketName":
                return output["OutputValue"]
        raise ValueError("BucketName not found in stack outputs")
    except Exception as e:
        print(f"Error getting bucket name: {e}")
        print("Make sure the RAGStack is deployed or set DOCUMENTS_BUCKET env
var.")
        sys.exit(1)


def parse_year_month(filepath):
    """Extract year and month from file path."""
    parts = filepath.parts
    year = None
    month_num = None
    month_name = None

    for part in parts:
        part_clean = part.strip()
        if part_clean in VALID_YEARS:
            year = part_clean
        part_lower = part_clean.lower()
        if part_lower in MONTH_NAMES:
            month_num = MONTH_NAMES[part_lower]
            month_name = MONTH_DISPLAY[month_num]

    return year, month_num, month_name


def discover_files(base_path, from_year=2020, to_year=2026):
    """Discover all PDF files organized by year/month."""
    files_by_period = defaultdict(list)
    unorganized_files = []
    skipped_files = []
```

```
    all_pdfs = list(base_path.glob("**/*.pdf"))

    for pdf_path in all_pdfs:
        year, month_num, month_name = parse_year_month(pdf_path)

        if year and month_num:
            year_int = int(year)
            if year_int < from_year or year_int > to_year:
                skipped_files.append((pdf_path, year, month_name, "outside year range"))
                continue
            files_by_period[(year, month_num, month_name)].append(pdf_path)
        else:
            unorganized_files.append(pdf_path)

    return dict(files_by_period), unorganized_files, skipped_files


def upload_file(s3_client, bucket, filepath, year=None, month=None, month_name=None):
    """Upload a single PDF file to S3 with FLAT structure."""
    file_uuid = str(uuid.uuid4())[:8]
    key = f"uploads/{file_uuid}_{filepath.name}"

    metadata = {
        "original-filename": sanitize_metadata(filepath.name),
        "original-path": sanitize_metadata(str(filepath)),
        "upload-id": file_uuid,
    }

    if year:
        metadata["source-year"] = year
    if month:
        metadata["source-month"] = month
    if month_name:
        metadata["source-month-name"] = month_name

    try:
        file_size = filepath.stat().st_size
        s3_client.upload_file(
            str(filepath),
            bucket,
            key,
            ExtraArgs={"ContentType": "application/pdf", "Metadata": metadata}
        )
        return {"file": filepath.name, "status": "success", "size": file_size, "year": year or "N/A"}
    except Exception as e:
        return {"file": filepath.name, "status": "error", "error": str(e), "year": year or "N/A"}
```

```
def format_size(size_bytes):
    """Format file size in human readable format."""
    for unit in ['B', 'KB', 'MB', 'GB']:
        if size_bytes < 1024:
            return f"{size_bytes:.1f} {unit}"
        size_bytes /= 1024
    return f"{size_bytes:.1f} TB"


def print_discovery_summary(files_by_period, unorganized, skipped, from_year, to_year):
    """Print summary of discovered files."""
    print("\n" + "=" * 70)
    print(f"DISCOVERY SUMMARY (Years {from_year}-{to_year})")
    print("=" * 70)

    total_size = 0
    total_files = 0

    if files_by_period:
        sorted_periods = sorted(files_by_period.keys(), key=lambda x: (x[0], x[1]))
        current_year = None
        year_file_count = 0
        year_size = 0

        for year, month_num, month_name in sorted_periods:
            if year != current_year:
                if current_year is not None:
                    print(f"    └── Year total: {year_file_count} file(s) ({format_size(year_size)})\n")
                print(f"📁 {year}/")
                current_year = year
                year_file_count = 0
                year_size = 0

            files = files_by_period[(year, month_num, month_name)]
            count = len(files)
            size = sum(f.stat().st_size for f in files)
            total_size += size
            total_files += count
            year_file_count += count
            year_size += size
            print(f"    ├── {month_name:<12} {count:>4} file(s) ({format_size(size):>10})")

        if current_year is not None:
            print(f"    └── Year total: {year_file_count} file(s) ({format_size(year_size)})")
```

```
        print(f"\n📊 Total organized files: {total_files} ({format_size(total_size)})")

    if unorganized:
        unorg_size = sum(f.stat().st_size for f in unorganized)
        print(f"\n⚠️  Unorganized files: {len(unorganized)} ({format_size(unorg_size)})")
        for f in unorganized[:5]:
            print(f"   - {f.name}")
        if len(unorganized) > 5:
            print(f"   ... and {len(unorganized) - 5} more")

    print(f"\n💾 Total size to upload: {format_size(total_size)}")
    print("=" * 70)

    return total_files, total_size


def upload_documents(path, dry_run=False, from_year=2020, to_year=2026,
include_unorganized=False):
    """Upload PDF documents from year/month directory structure to FLAT S3 structure."""
    base_path = Path(path)

    if not base_path.exists():
        print(f"Error: Path does not exist: {base_path}")
        sys.exit(1)

    if not base_path.is_dir():
        print(f"Error: Path is not a directory: {base_path}")
        sys.exit(1)

    print(f"🔍 Scanning directory: {base_path}")
    print(f"   Looking for: years {from_year}-{to_year}, all 12 months")

    files_by_period, unorganized_files, skipped_files = discover_files(base_path, from_year,
to_year)

    if not files_by_period and not unorganized_files:
        print("\nNo PDF files found matching criteria.")
        return

    total_files, total_size = print_discovery_summary(
        files_by_period, unorganized_files, skipped_files, from_year, to_year
    )

    if include_unorganized:
        total_files += len(unorganized_files)
```

```
    if total_files == 0:
        print("\nNo files to upload with current filters.")
        return

    if dry_run:
        print(f"\n 🔸 DRY RUN - Would upload {total_files} file(s)")
        return

    print(f"\n📤 Ready to upload {total_files} file(s)")
    response = input("Proceed? [y/N]: ").strip().lower()
    if response != 'y':
        print("Upload cancelled.")
        return

    bucket = get_bucket_name()
    print(f"\n☁️  Uploading to: s3://{bucket}/uploads/")

    s3_client = boto3.client("s3")

    # Build upload list
    upload_tasks = []
    for (year, month_num, month_name), files in files_by_period.items():
        for filepath in files:
            upload_tasks.append({"filepath": filepath, "year": year, "month": month_num,
"month_name": month_name})

    if include_unorganized:
        for filepath in unorganized_files:
            upload_tasks.append({"filepath": filepath, "year": None, "month": None,
"month_name": None})

    # Upload sequentially
    results = []
    start_time = time.time()
    successful = 0
    failed = 0
    total_uploaded_size = 0

    print(f"\nUploading {len(upload_tasks)} files...\n")

    for i, task in enumerate(upload_tasks, 1):
        result = upload_file(s3_client, bucket, task["filepath"], task["year"], task["month"],
task["month_name"])
        results.append(result)
```

```
        if result["status"] == "success":
            successful += 1
            total_uploaded_size += result.get("size", 0)
            icon = "✓"
        else:
            failed += 1
            icon = "✗"
            print(f"   {icon} {result['file']}: {result.get('error', 'Unknown error')}")

        progress = (i / len(upload_tasks)) * 100
        print(f"\r[{progress:5.1f}%] {successful} uploaded, {failed} failed", end="", flush=True)

    elapsed = time.time() - start_time

    # Summary
    print("\n\n" + "=" * 70)
    print("UPLOAD COMPLETE")
    print("=" * 70)
    print(f"⏱️  Duration: {elapsed:.1f} seconds")
    print(f"✅ Successful: {successful} ({format_size(total_uploaded_size)})")
    print(f"❌ Failed: {failed}")

    if failed > 0:
        print("\nFailed files:")
        for result in results:
            if result["status"] == "error":
                print(f"   - {result['file']}: {result['error']}")

    print("=" * 70)


def main():
    parser = argparse.ArgumentParser(description="Upload PDF documents to RAG
system")
    parser.add_argument("path", help="Path to directory containing year/month structured
PDFs")
    parser.add_argument("--dry-run", action="store_true", help="Preview without
uploading")
    parser.add_argument("--year", type=int, choices=range(2020, 2027), metavar="YEAR",
help="Upload only specific year")
    parser.add_argument("--from-year", type=int, default=2020, metavar="YEAR",
help="Start year (default: 2020)")
    parser.add_argument("--to-year", type=int, default=2026, metavar="YEAR", help="End
year (default: 2026)")
    parser.add_argument("--include-unorganized", action="store_true", help="Include files
not in year/month structure")
```

```
    args = parser.parse_args()

    from_year = args.from_year
    to_year = args.to_year

    if args.year:
        from_year = args.year
        to_year = args.year

    if from_year > to_year:
        print("Error: --from-year cannot be greater than --to-year")
        sys.exit(1)

    upload_documents(
        path=args.path,
        dry_run=args.dry_run,
        from_year=from_year,
        to_year=to_year,
        include_unorganized=args.include_unorganized
    )


if __name__ == "__main__":
```

Test the script

Before loading the files to S3, one may want to test the script with the following option:

$ python scripts/upload_documents_fixed.py ./datasets --dry-run

Part II: Deployment Guide

The development workstation runs Ubuntu and requires Python 3.9, Node.js 18+, the AWS CLI v2, Docker (used by CDK for asset bundling), and the AWS CDK CLI installed globally via `npm install -g aws-cdk`. AWS credentials are configured via `aws configure` after generating an access key pair from the IAM console; the recommended verification command is `aws sts get-caller-identity`, which should return the account ID and IAM ARN. If that call returns `InvalidClientTokenId`, the access key has either been rotated or never propagated, and a new key must be generated.

Prerequisites on Ubuntu PC

Step 1: Install Required Software

```
# Update package list
sudo apt update

# Install Python 3.9  - must match Lambda runtime
sudo apt install -y software-properties-common
sudo add-apt-repository -y ppa:deadsnakes/ppa
sudo apt update
sudo apt install -y python3.9 python3.9-venv python3.9-dev

# Install Node.js 18+
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs

# Install AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
rm -rf awscliv2.zip aws/


# Install unzip if not present
 sudo apt install -y unzip
```

Step 2: Install AWS CDK

```
sudo npm install -g aws-cdk
cdk --version
```

Step 3: Configure AWS Credentials

```
aws configure
```

Enter your AWS Access Key ID, Secret Access Key, default region (us-east-1), and output format (json).

verify credentials:

```
aws sts get-caller-identity
```

Prerequisites in AWS Console

In the Bedrock console, model access must be explicitly granted for "Amazon Titan Embeddings V2" and for the chosen Claude model. Newer Claude models such as Claude Sonnet 4.5 are only accessible through an *inference profile* (the `us.anthropic.…` prefix), not as a bare foundation-model ARN.

Step 4: Enable Bedrock Model Access

Log into the AWS Console and navigate to Amazon Bedrock, then Model access. Request access to "Amazon Titan Embeddings V2" and "Anthropic Claude 4.5 Sonnet".

To find available models:

```
aws bedrock list-foundation-models --by-provider anthropic \
  --query "modelSummaries[*].[modelId,modelLifecycle.status]" --output table
```

Select a model with status ACTIVE. As of this writing, the working model ID is `us.anthropic.claude-sonnet-4-5-20250929-v1:0`.

Create the Project

Step 1: Create Directory Structure

```
mkdir -p rag-solution/{config,stacks,scripts}
mkdir -p rag-solution/lambdas/{document_processor,index_manager,query_handler}
mkdir -p rag-solution/layers/dependencies/python/
cd rag-solution
```

Step 2: Create All Files

Create each file from Part I above.

Step 3: Create requirements.txt Files

Project root requirements.txt:

```
aws-cdk-lib==2.170.0
constructs>=10.0.0,<11.0.0
python-dotenv==1.0.0
boto3==1.34.0
```

layers/dependencies/requirements.txt:

```
faiss-cpu==1.7.4
numpy==1.26.4
pypdf>=5.0.0
```

Step 4: Install Lambda Layer Dependencies

This step is critical. Lambda runs on Amazon Linux with Python 3.9, so dependencies (NumPy, FAISS) must be compiled for that exact platform and Python minor version.

```
cd layers/dependencies
rm -rf python/*

pip install \
  --platform manylinux2014_x86_64 \
  --implementation cp \
  --python-version 3.9 \
  --only-binary=:all: \
  --target python/ \
  numpy==1.26.4 faiss-cpu==1.7.4 “pypdf>=5.0.0”

# Verify the installation
ls python/
```

Quote `"pypdf>=5.0.0"` to prevent the shell from interpreting `>` as output redirection. The `--only-binary=:all:` flag is also critical: without it, pip may silently fall back to a source build that compiles against the Ubuntu host toolchain and reintroduces the NumPy C-extension incompatibility inside Lambda.

The four flags (`--platform`, `--implementation`, `--python-version`, `--only-binary`) collectively force `pip` to download the precompiled Linux wheel that matches the Lambda runtime rather than building from source against the local interpreter. The Lambda function definitions in `rag_stack.py` accordingly specify `lambda_.Runtime.PYTHON_3_9`.

Step 5: Set Up Python Virtual Environment

```
cd ~/rag-solution
python3.9 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
```

Step 6: Set Environment Variables

```
# Get your AWS account ID
AWS_ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
echo "Your AWS Account ID: $AWS_ACCOUNT"

# Create .env file
cat > .env << EOF
CDK_DEFAULT_ACCOUNT=$AWS_ACCOUNT
CDK_DEFAULT_REGION=us-east-1
EOF

# Load environment variables
export $(cat .env | xargs)
```

Step 7: Configure the LLM Model ID

Edit `stacks/rag_stack.py` and set the correct model ID in the Lambda environment variables:

"LLM_MODEL_ID": "us.anthropic.claude-sonnet-4-5-20250929-v1:0",

Deploy the Solution

Step 1: Bootstrap CDK (first time only)

The Python virtual environment is created and populated with the CDK dependencies (`aws-cdk-lib==2.170.0`, `constructs>=10.0.0,<11.0.0`, `boto3==1.34.0`, `python-dotenv==1.0.0`).

```
cd ~/rag-solution
source .venv/bin/activate
cdk bootstrap aws://$CDK_DEFAULT_ACCOUNT/$CDK_DEFAULT_REGION
```

Step 2: Synthesize the CloudFormation Template

cdk synth

This command produces the CloudFormation template.

Step 3: Deploy the Stack

cdk deploy

This command provisions the stack.

Output it like :

```
RAGStack.APIEndpoint = https://xxxxxxxxxx.execute-api.us-east-1.amazonaws.com/prod/
RAGStack.BucketName = ragstack-ragbucketxxxxxxxx-xxxxxxxxxxxx
```

Step 4: Verify

```
# Get stack outputs
aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs" --output table

# List Lambda functions
```

```
aws lambda list-functions --query "Functions[?contains(FunctionName,
'RAGStack')].FunctionName" --output table
```

Upload Documents and Build the Index

Step 1: Get Bucket Name

```
BUCKET=$(aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs[?contains(OutputKey, 'Bucket')].OutputValue" --output text)
echo "Bucket: $BUCKET"
```

Step 2: Upload Test Document

```
# Create a test document
echo "Amazon Bedrock is a fully managed service that offers foundation models from
leading AI companies. It provides access to models from Anthropic, AI21 Labs, Stability
AI, and Amazon. You can use it for text generation, summarization, and embeddings." >
test-doc.txt

# Upload to S3
aws s3 cp test-doc.txt s3://$BUCKET/uploads/

# Verify upload
aws s3 ls s3://$BUCKET/uploads/
```

Step 3: Process the Document

Manual processing:

```
PROCESSOR=$(aws lambda list-functions \
  --query "Functions[?contains(FunctionName, 'DocumentProcessor')].FunctionName"
--output text)
```

```
aws lambda invoke \
  --function-name $PROCESSOR \
  --payload '{"bucket": "'$BUCKET'", "key": "uploads/test-doc.txt"}' \
  --cli-binary-format raw-in-base64-out \
  response.json

cat response.json
```

Step 4: Build the FAISS Index

This step is required after uploading documents. The index must be rebuilt to make documents searchable.

```
INDEX_MANAGER=$(aws lambda list-functions \
  --query "Functions[?contains(FunctionName, 'IndexManager')].FunctionName" --output
text)

aws lambda invoke \
  --function-name $INDEX_MANAGER \
  --payload '{}' \
  --cli-binary-format raw-in-base64-out \
  --cli-read-timeout 300 \
  response.json

cat response.json
```

expected output:

{"statusCode": 200, "body": "{\"message\": \"Index built successfully\", \"total_vectors\": 1, \"dimension\": 1024}"}

Step 4.5: Retrieve the API Key

The stack creates an API key that must be sent in the `x-api-key` header on every request. The key ID is exported as a CloudFormation output (`ApiKeyId`), but the key *value* must be retrieved separately:

```
# Get the key ID from stack outputs
KEY_ID=$(aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs[?OutputKey=='ApiKeyId'].OutputValue" \
```

```
  --output text)

# Retrieve the actual key value
API_KEY=$(aws apigateway get-api-key --api-key $KEY_ID --include-value \
  --query "value" --output text)

echo "API Key: $API_KEY"
```

Store this value securely — treat it as a secret. For repeated use, export it in your shell profile:

```
export RAG_API_KEY="<the-key-value>"
```

Step 5: Test Query

```
API_ENDPOINT=$(aws cloudformation describe-stacks --stack-name RAGStack \
  --query "Stacks[0].Outputs[?contains(OutputKey, 'APIEndpoint')].OutputValue" \
  --output text)

curl -X POST ${API_ENDPOINT}query \
  -H "Content-Type: application/json" \
  -H "x-api-key: $RAG_API_KEY" \
  -d '{"query": "What is Amazon Bedrock?"}'
```

Upload PDF Documents

Step 1: Install Script Dependencies

No additional dependencies required.

Step 2: Upload Script

The source code is in Part 1 of this guide.

The code scripts/upload_documents_fixed.py traverses a local directory tree organised as `datasets/<year>/<month>/*.pdf`, validates that years fall in the supported 2020–2026

range, and uploads each file with a unique key of the form `uploads/<8-char-uuid>_<filename>.pdf`. Original path information is preserved in the S3 object metadata (`original-filename`, `original-path`, `source-year`, `source-month`, `source-month-name`).

S3 metadata must be ASCII. Many real-world article titles contain typographic Unicode characters — curly apostrophes ('), em-dashes (–), accented letters — and any of these in a metadata value causes `boto3` to raise a `Parameter validation failed: Non ascii characters found in S3 metadata` error. The misleading aspect of this failure is that the error surfaces during the PUT call and is often misreported as a "malformed PDF" problem.

Step 3: Upload Documents

python scripts/upload_documents_fixed.py /path/to/pdf/files $BUCKET 2020

Step 4: Rebuild the Index

```
INDEX_MANAGER=$(aws lambda list-functions \
  --query "Functions[?contains(FunctionName, 'IndexManager')].FunctionName" \
  --output text)

echo "Target function: $INDEX_MANAGER" # sanity check — must not be empty


aws lambda invoke \
  --function-name $INDEX_MANAGER \
  --payload '{}' \
  --cli-binary-format raw-in-base64-out \
  --cli-read-timeout 600 \
  response.json

cat response.json
aws s3 ls s3://$BUCKET/faiss-index/
```

The `echo` sanity check is important: if the `list-functions` query runs before `cdk deploy` has finished propagating, `$INDEX_MANAGER` will be empty and `aws lambda invoke` will silently target nothing, leaving the FAISS index unbuilt. Subsequent queries then fail with a 404 on `HeadObject` because `query_handler.load_index()` cannot find `index.faiss` under the `faiss-index/` prefix.

Once the bucket trigger has been added to the stack (`self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED, s3n.LambdaDestination(self.document_processor), s3.NotificationKeyFilter(prefix="uploads/"))`), only *new* uploads cause the Document Processor to run. Files that were already in the bucket before the trigger was wired up will sit there untouched, and CloudWatch will show no invocations. The only remedies are to either delete and re-upload the affected objects, or to invoke the processor manually for each existing key.

Step 5: Verify the Index

```
# Check number of indexed documents
aws s3 cp s3://$BUCKET/faiss-index/metadata.json - | \
  python3 -c "import sys,json; d=json.load(sys.stdin); \
  print('Total vectors:', len(d)); \
  sources=set(item.get('source','') for item in d); \
  print('Unique sources:', len(sources))"
```

Updating dependencies and re-ingesting the corpus.

When a Lambda layer dependency must be upgraded (as happened with pypdf 3.17.0 → ≥ 5.0.0), the change propagates through four stages that must be applied in order: (1) update `layers/dependencies/requirements.txt`; (2) rebuild the layer via the `pip install` command in Step 4 of the deployment guide; (3) redeploy with `cdk deploy` to publish the new layer version; (4) wipe the three S3 data prefixes (`faiss-index/`, `processed/`, `uploads/`) and re-ingest the corpus. All three prefixes must be wiped because `uploads/` holds the source PDFs that trigger DocumentProcessor, `processed/` holds the intermediate chunks that IndexManager reads, and `faiss-index/` holds the searchable index itself; leaving any one of them populated with pre-upgrade output will mix stale and fresh data in the next rebuild.

Commands and scripts

- to delete : `aws s3 rm --recursive`
- to load files : do_load_0.sh
    - #!/bin/bash
    - FUNCTION_NAME=$(aws lambda list-functions --query "Functions[?contains(FunctionName, 'IndexManager')].FunctionName" --output text)
    -

- for year in 2020 2021 2022 2023 2024 2025 2026; do
- python scripts/upload_documents_fixed.py /media/joepareti54/Elements/x/finance-${year}/press --year $year
- done
-
- # Rebuild index after all uploads complete
- aws lambda invoke --function-name $FUNCTION_NAME --payload '{}' --cli-binary-format raw-in-base64-out --cli-read-timeout 600 response.json
-
-

Troubleshooting

Refer to this document that explains my own implementation. Below some key points are summarized.

Lambda Layer

Lambda runs on Amazon Linux 2 with a specific Python interpreter, and any compiled C extension (which includes both NumPy and FAISS) must be built for that exact platform and Python minor version. Installing packages with a plain `pip install -t python/` on an Ubuntu workstation produces wheels that fail at import time inside Lambda with `No module named 'numpy.core._multiarray_umath'`.

To make it work do:

```
cd layers/dependencies
rm -rf python/*
pip install \
  "numpy==1.26.4" \
  "faiss-cpu==1.7.4" \
  "pypdf>=5.0.0" \
  -t python/ \
  --platform manylinux2014_x86_64 \
  --implementation cp \
  --python-version 3.9 \
  --only-binary=:all:
```

PDF Parsing

The original design used PyMuPDF (`fitz`), which has complex native dependencies and produces a misleading `ImportError: cannot import name '_extra' from partially initialized module 'fitz'` inside Lambda. Switching to the pure-Python `pypdf` library eliminated the problem at the cost of less robust text extraction.

pypdf 3.17.0 → ≥ 5.0.0. Running the system against the full 2020–2026 corpus revealed that pypdf 3.17.0 produced corrupted text in some chunks, visible in the `text` field of verbose query responses. The corruption was reproducible locally against affected PDFs and resolved by upgrading to pypdf ≥ 5.0.0. Any new deployment should install the newer release from the outset; existing deployments must rebuild the Lambda layer, redeploy via `cdk deploy`, wipe `faiss-index/`, `processed/`, and `uploads/` in S3, and re-ingest the corpus. See the companion document "Updating the AWS/LLM/RAG system" for the full remediation procedure.

When uploading files to S3, metadata must be ASCII

Many article titles contain typographic Unicode characters — curly apostrophes (’), em-dashes (–), accented letters — and any of these in a metadata value causes `boto3` to raise a `Parameter validation failed: Non ascii characters found in S3 metadata` error. The misleading aspect of this failure is that the error surfaces during the PUT call and is often misreported as a "malformed PDF" problem. The script has been reworked to fix the issue.

S3 event notifications are not retroactive

Once the bucket trigger has been added to the stack (`self.bucket.add_event_notification(s3.EventType.OBJECT_CREATED, s3n.LambdaDestination(self.document_processor), s3.NotificationKeyFilter(prefix="uploads/"))`), only *new* uploads cause the Document Processor to run. Files that were already in the bucket before the trigger was wired up will sit there untouched, and CloudWatch will show no invocations. The only remedies are to either delete and re-upload the affected objects, or to invoke the processor manually for each existing key.

Concurrency throttling under bulk ingest

An AWS account ships with a limit of **10 concurrent Lambda executions**. Uploading 3,477 PDFs may saturate that limit: every S3 PUT fires a Document Processor invocation, and

once concurrency is exhausted both new invocations *and* the user-facing Query Handler are throttled with `TooManyRequestsException: Rate Exceeded`. From the outside this presents as `Internal server error` on the `/query` endpoint, while the document corpus appears to load successfully.

403 Forbidden on /query

this exception likely indicates an API key problem. 3 common causes are:

- missing x-api-key header
- whitespace in the key value
- recent rotation (if any) still propagating

Indexing and S3 List Pagination

After the full 2020–2026 corpus was uploaded and the index was rebuilt, the index reported **15,812 vectors**. This number is suspicious: the 2020-only index had contained 8,093 vectors, and with seven years of roughly equal-sized data one would expect something closer to 50,000 vectors. A direct count of unique sources in the metadata file revealed the truth

```
aws s3 cp s3://<bucket>/faiss-index/metadata.json - | python3 -c \
  "import sys,json; d=json.load(sys.stdin); \
   print('Total entries:', len(d)); \
   print('Unique sources:', len(set(item.get('source','') for item in d)))"

Total entries: 15812
Unique sources: 1000
```

Exactly 1,000 unique source documents — which is the precise default `MaxKeys` value of the S3 `list_objects_v2` API. The original `index_manager.py` made a single un-paginated call:

```
response = s3.list_objects_v2(Bucket=BUCKET_NAME, Prefix=processed_prefix)
for obj in response.get("Contents", []):
    ...
```

This caps the index at the first 1,000 processed files **without warnings**; the remaining 2,281 documents in the corpus were never embedded into the searchable index. The fix is to use the paginator:

```
paginator = s3.get_paginator('list_objects_v2')
pages = paginator.paginate(Bucket=BUCKET_NAME, Prefix=processed_prefix)
for page in pages:
    for obj in page.get("Contents", []):
```

After deploying the corrected `index_manager.py` and rebuilding, the index grew to **50,727 vectors across 3,281 unique sources**, matching the number of processed files. Subsequent queries about post-pandemic monetary policy returned answers grounded in sources from across all seven years (Barron's and WSJ articles dated 2022 through 2025), confirming that the full corpus was now searchable.

CLI Flag

Why `--cli-binary-format raw-in-base64-out` and `--cli-read-timeout 600` are both required. Lambda's `Invoke` API models `Payload` as a binary blob. AWS CLI v2 changed the default so that any blob-typed argument is treated as base64-encoded on input. Passing `--payload '{}'` without the flag causes the CLI to attempt to base64-decode `{}` — which is not valid base64 — producing either a decode error or two garbage bytes that Lambda cannot parse as JSON. The value `raw-in-base64-out` restores v1 behaviour for inputs (string sent verbatim as payload bytes) while keeping the safer v2 behaviour for outputs (binary in the response is base64-encoded so it doesn't corrupt `response.json` or the terminal). The `--cli-read-timeout 600` is required because IndexManager's rebuild of the full corpus can run well beyond the default CLI timeout of 60 seconds.

Running Test Queries

Archegos and Credit Suisse

curl -sS -X POST "${API_ENDPOINT}/query" -H "Content-Type: application/json" -H "x-api-key: $RAG_API_KEY" -d '{"query": "**Review the Archegos failure in the context of total return swaps and collateral mandated by Credit Suisse"**, "k": 15}'
{"query": "Review the Archegos failure in the context of total return swaps and collateral mandated by Credit Suisse", "answer": "The Archegos failure involved significant issues with total return swaps and collateral management by Credit Suisse. Credit Suisse progressively weakened its collateral requirements for Archegos over time. In 2019, Archegos successfully negotiated to lower its margin requirement on total return swaps from around 20% to just 7.5% of the total invested, claiming competitors were offering better deals. In exchange, Archegos agreed to give Credit Suisse more power to close out positions with little notice,

but the report found these protections were \"illusory, as the business appears to have had no intention of invoking them for fear of alienating the client.\"\n\nA critical failure occurred in mid-March 2021 when Credit Suisse made a \"fateful decision\" to return $2.4 billion in margin collateral to Archegos between March 11 and March 19. This happened after the value of Archegos's holdings in stocks like ViacomCBS and Discovery had skyrocketed. Credit Suisse had initially calculated that Archegos would need to put up an additional $1.4 billion in margin and wanted to implement a new system incorporating volatility and position concentration into margin calculations. However, Archegos canceled calls to discuss this and instead requested margin back.\n\nThe collateral return was problematic because Archegos held concentrated positions in a handful of stocks rather than a diverse portfolio, creating special risks. By March 23, Credit Suisse's gross exposure to Archegos had grown to $27 billion. When ViacomCBS shares fell on March 22, Archegos couldn't make margin calls and entered a downward spiral, ultimately resulting in over $5.5 billion in losses for Credit Suisse. In contrast, some of Credit Suisse's rivals noticed the increasing concentration risk and moved in the opposite direction, demanding Archegos back up its investments with additional cash.", "sources": [{"index": 0, "chunk_id": "CHUNK_0", "source": "uploads/7d389f93_Archegos Blowout Heaps Pressure on Credit Suisse - WSJ.pdf", "score": 0.6550253033638}, {"index": 1, "chunk_id": "CHUNK_1", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.6510246992111206}, {"index": 2, "chunk_id": "CHUNK_2", "source": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "score": 0.6486321687698364}, {"index": 3, "chunk_id": "CHUNK_3", "source": "uploads/b846e9b8_Inside Archegos\u2019s Epic Meltdown - WSJ.pdf", "score": 0.6385998725891113}, {"index": 4, "chunk_id": "CHUNK_4", "source": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "score": 0.6330972909927368}, {"index": 5, "chunk_id": "CHUNK_5", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.6316956281661987}, {"index": 6, "chunk_id": "CHUNK_6", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.6258243322372437}, {"index": 7, "chunk_id": "CHUNK_7", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.6189457774162292}, {"index": 8, "chunk_id": "CHUNK_8", "source": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "score": 0.6105198264122009}, {"index": 9, "chunk_id": "CHUNK_9", "source": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "score": 0.6002570986747742}, {"index": 10, "chunk_id": "CHUNK_10", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.5964667797088623}, {"index": 11, "chunk_id": "CHUNK_11", "source": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "score": 0.5832305550575256}, {"index": 12, "chunk_id": "CHUNK_12", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.5827152729034424}, {"index": 13, "chunk_id": "CHUNK_13", "source": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "score": 0.5820020437240601}, {"index": 14, "chunk_id": "CHUNK_14", "source": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "score": 0.5819920301437378}], "retrieval_status": "sources_used", "sources_used": [{"chunk_id": "CHUNK_1", "source_file": "uploads/76b3fbb5_Credit Suisse Failed to Act on

Archegos Risks, Report Says - WSJ.pdf", "facts_extracted": ["Margin on total return swaps dropped to 7.5% from around 20% in 2019", "In return for lower margins, Archegos gave Credit Suisse more power to close out positions with little notice", "These protections were illusory as the business had no intention of invoking them for fear of alienating the client"]}, {"chunk_id": "CHUNK_2", "source_file": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "facts_extracted": ["Credit Suisse released margin payments back to the fund", "Returning collateral was problematic because Archegos held concentrated positions in a handful of stocks", "On March 22, ViacomCBS shares fell and Archegos couldn't make margin calls"]}, {"chunk_id": "CHUNK_5", "source_file": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "facts_extracted": ["Credit Suisse calculated Archegos would need to put up an additional $1.4 billion margin in mid-March", "Credit Suisse wanted to implement a new system incorporating volatility and position concentration into margin calculations", "Credit Suisse returned $2.4 billion in margin collateral to Archegos between March 11 and March 19", "By March 23, Credit Suisse's gross exposure to Archegos was $27 billion"]}, {"chunk_id": "CHUNK_9", "source_file": "uploads/065a0698_Inside Credit Suisse\u2019s $5.5 Billion Breakdown - WSJ.pdf", "facts_extracted": ["Archegos had handed over cash to Credit Suisse to secure its bets as standard practice", "With stocks more than doubling, Archegos asked for money back and it was credited", "The transfer meant Archegos had even less cash backing up its positions", "Credit Suisse's rivals moved in the opposite direction, demanding additional cash due to concentration risk"]}, {"chunk_id": "CHUNK_13", "source_file": "uploads/76b3fbb5_Credit Suisse Failed to Act on Archegos Risks, Report Says - WSJ.pdf", "facts_extracted": ["In 2017, a 10% margin call was waived by Credit Suisse", "Credit Suisse created bespoke weekly monitoring of Archegos instead", "In 2019, margin on total return swaps dropped from around 20% to 7.5%"]}], "information_gaps": []}(.venv) joepareti54@joepareti54-GS75-Stealth-9SE:/media/joepareti54/Elements/x/finance-2026(.venv) joepareti54@joepareti

Private Credit - Irrational Exuberance

curl -sS -X POST "${API_ENDPOINT}/query" -H "Content-Type: application/json" -H "x-api-key: $RAG_API_KEY" -d '{"query": **"review trends and systemic risks posed by private credit in the context of irrational exhuberance in several financial markets. if irrational exhuberance is not in context , please complement using plausible general financial knoweldge"**}'
{"query": "review trends and systemic risks posed by private credit in the context of irrational exhuberance in several financial markets. if irrational exhuberance is not in context , please complement using plausible general financial knoweldge", "answer": "The retrieved documents highlight several significant systemic risks and trends in private credit markets that align with concerns about excessive optimism, though the term 'irrational exuberance' is not explicitly used.\n\n**Key Trends in Private Credit:**\n1. **Rapid Growth**: The sector has grown dramatically over the past decade, expanding faster than regulatory oversight can track. Jamie Dimon noted in 2023 that private credit is pushing economic activity 'out of the sight of regulators.'\n2. **Retail Investor Expansion**: The industry is actively courting retail investors, which could create channels for risks to seep into broader financial markets

through confidence issues.\n3. **Deteriorating Credit Quality**: Leverage metrics have deteriorated and covenant protections have weakened as competition for deals has intensified in the growing private credit market.\n4. **Sector Concentration**: Many loans are concentrated in software, business services, and healthcare, particularly in companies backed by private-equity funds.\n\n**Systemic Risks Identified:**\n1. **Untested in Downturns**: As noted by Fabio Natalucci of the Andersen Institute, 'We have not seen this sector, at this size, go through a meaningful slowdown.' The sector's systemic risk to the economy 'isn't fundamentally understood because of how fast they have grown.'\n2. **Default Rate Concerns**: While the industry anticipates default rates of 5-6%, experts worry expectations may be 'too rosy,' based on artificially low pandemic-era default rates when the Fed provided trillions in stimulus. With the Fed now raising rates and trimming its balance sheet, such assistance is unlikely to be repeated.\n3. **Interest Coverage Stress**: Companies are generating less cash flow while facing higher interest rates, affecting their ability to cover interest payments. Some borrowers may be at risk of 'running out of money.'\n4. **Potential Contagion**: A combination of rising defaults, elevated interest rates, and flagging investor appetite could exacerbate a downturn, though 'in slow motion, given the nature of borrowing.'\n5. **Regulatory Gap**: Private-markets companies are 'imitating banks but don't face the same regulations.'\n\n**Complementing with Financial Knowledge on Irrational Exuberance Indicators:**\nThe documented trends suggest several classic signs of irrational exuberance: (1) weakening underwriting standards (deteriorating leverage metrics and covenant protections), (2) rapid growth outpacing risk management infrastructure, (3) expansion into retail markets during peak growth, (4) excessive optimism about default rates despite changing macroeconomic conditions, and (5) Jamie Dimon's May 2024 warning that private credit's 'happy-go-lucky' growth hasn't been tested and there could 'be hell to pay' if loans sour with everyday investors 'holding the bag.' His characterization of credit as a 'bad risk' today echoes concerns about mispricing of risk typical of exuberant markets.", "sources": [{"index": 0, "chunk_id": "CHUNK_0", "source": "uploads/84e6ff41_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "score": 0.5271496772766113}, {"index": 1, "chunk_id": "CHUNK_1", "source": "uploads/2972a79a_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "score": 0.49635636806488037}, {"index": 2, "chunk_id": "CHUNK_2", "source": "uploads/26c711a8_Jamie Dimon Says Private Credit Is Dangerous\u2014and He Wants JPMorgan to Get In on It - WSJ.pdf", "score": 0.49418920278549194}, {"index": 3, "chunk_id": "CHUNK_3", "source": "uploads/84e6ff41_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "score": 0.4893418848514557}, {"index": 4, "chunk_id": "CHUNK_4", "source": "uploads/2972a79a_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "score": 0.4893418848514557}], "retrieval_status": "sources_used", "sources_used": [{"chunk_id": "CHUNK_0", "source_file": "uploads/84e6ff41_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "facts_extracted": ["Redemption requests fell in first quarter but concerns exist about smaller players", "Industry courting retail investors could increase risks seeping into broader markets", "Anticipated default rates of 5-6% may be too rosy based on low pandemic default rates with Fed stimulus", "Fed raising rates and trimming balance sheet - assistance unlikely to be repeated", "Leverage metrics have deteriorated", "Covenant protections have weakened as growth increased competition for

deals"]}, {"chunk_id": "CHUNK_1", "source_file": "uploads/2972a79a_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "facts_extracted": ["Anticipated default rates of 5-6%", "Leverage metrics have deteriorated", "Many loans concentrated in software, business services, and healthcare in companies backed by private-equity funds"]}, {"chunk_id": "CHUNK_2", "source_file": "uploads/26c711a8_Jamie Dimon Says Private Credit Is Dangerous\u2014and He Wants JPMorgan to Get In on It - WSJ.pdf", "facts_extracted": ["Private-markets companies imitating banks but don't face same regulations", "Systemic risk not fundamentally understood due to fast growth", "Sector has not gone through meaningful slowdown at this size", "Dimon told Congress in 2023 private credit pushing activity out of sight of regulators", "Dimon warned in 2024 could be 'hell to pay' if loans sour and everyday investors left holding the bag", "Dimon said in May credit is a 'bad risk' and 'happy-go-lucky' growth hasn't been tested in downturn"]}, {"chunk_id": "CHUNK_3", "source_file": "uploads/84e6ff41_\u2018Shadow Banks\u2019 Hold Half of the World\u2019s Assets. Why Concerns Are Growing. - Barrons.pdf", "facts_extracted": ["Companies generating less cash flow affecting ability to cover interest payments", "Leverage not as high as financial crisis limiting potential systemic risk", "Risks transferred to individual borrower", "Concern about borrowers running out of money", "Combination of rising defaults, elevated interest rates, and flagging investor appetite could exacerbate downturn in slow motion", "Much direct lending involves senior secured debt which should minimize losses"]}], "information_gaps": ["The term 'irrational exuberance' is not explicitly mentioned in the retrieved documents", "Specific quantitative data on total market size or growth rates", "Specific margin or collateral requirements in private credit markets", "Comparison of current leverage ratios to historical benchmarks beyond 'not as high as financial crisis'", "Specific default rate data beyond the anticipated 5-6% figure"]}(.venv) joepareti54@joepareti5(.venv) joepareti54@joepareti54-GS75-Stealth-9SE:/media/joepareti54/Elements/x/finance-2026/AI/Agent_AI_AWS/rag-solution$

Cost

The FAISS-on-S3 architecture is cheaper than the OpenSearch-based alternative originally considered. Storing the index in S3 costs cents per month rather than the $70–$100/month of a managed OpenSearch domain, and no NAT Gateway is required since all Lambdas reach Bedrock and S3 over the AWS network. Total fixed infrastructure cost is about $5–$10/month, with Bedrock invocation costs scaling linearly with usage including the Titan embedding calls during indexing, and the Claude calls during querying.

Component	OpenSearch Solution	FAISS Solution
Vector Store	~$70-100/month	~$0.50/month (S3)
NAT Gateway	~$30-45/month	$0 (not needed)
Lambda	~$5/month	~$5/month
Bedrock	Usage-based	Usage-based
Total	**~$105-150/month**	**~$5-10/month**

Results and Discussion

The serverless RAG system processes approximately 3,300 financial press articles spanning 2020–2026, producing a FAISS index of 50,727 vectors across 3,281 unique sources. Query latency typically ranges from 3–8 seconds, dominated by the Claude Sonnet 4.5 inference call rather than FAISS retrieval.

Github

The code is available here.

Retrieval Quality and Groundedness

Test queries demonstrate semantically relevant retrieval across the full temporal range of the corpus, with cross-language behavior (German sources appearing for English queries) confirming Titan v2's multilingual capabilities. The system exhibits appropriate grounding: when presented with false premises, the model explicitly contradicts the user rather than echoing the assertion, and when information is absent from retrieved context, it acknowledges the gap.

Comparison with the Local Llama 2 Prototype

The AWS implementation eliminates the constraints that shaped the earlier prototype: 6 GB VRAM limitation, 4-bit quantization, low-dimensional embeddings, document truncation to 1,000 characters, and the two-step summarize-then-answer prompt strategy. The two-step approach has been replaced with single-step retrieved-context prompting, since Claude on Bedrock has no memory constraint analogous to the local system.

Features Proposed but not implemented

Several enhancements from the Llama 2 prototype document remain unimplemented:

- **Metadata-aware retrieval with date weighting** — the proposed scoring formula final_score=α · (1−d)+β · recency is not implemented; retrieval is purely by embedding similarity.
- **Time-sensitivity filtering** — no automatic detection of temporal phrases like "in recent years."
- **Relevance threshold with fallback** — the system always passes top-k chunks regardless of similarity score.
- **Timestamp disclaimers** — not systematically enforced in responses.
- **Chunk-based retrieval** — partially addressed; the Document Processor chunks PDFs into ~1,000-char segments with 200-char overlap, but within-document pivoting windows are not implemented.

Toward Agentic AI in AWS

This project serves as a stepping stone toward an agentic AI implementation using AWS Bedrock and other AWS components.. The current architecture is a single-turn RAG pipeline; an agentic extension would add query planning, iterative retrieval, multi-step reasoning with memory, and tool integration (live data feeds, calculation engines, citation verification). The cost and latency characteristics of the FAISS-on-S3 architecture—index cached in Lambda memory, stateless design enabling parallel invocations—make it well suited as a retrieval tool within an agentic loop.

Limitations

The system lacks incremental index updates (rebuilding the full index takes 2–3 minutes for 3,300 documents), and does not implement confidence scoring or answer verification.